Lezione 8 - L'incapsulamento

Introduzione

Finora abbiamo visto come definire classi in Java, come creare **oggetti** tramite i costruttori e come gestire la visibilità**e il comportamento dei membri** di una classe attraverso i **modificatori** (`public` , `private` , `protected` , `final` , `static`). Abbiamo approfondito concetti chiave come:

- **Overloading dei metodi e dei costruttori**, che **permette di fornire più versioni dello stesso metodo o costruttore con parametri diversi, garantendo flessibilità nella creazione degli oggetti.**
- **Il ruolo della keyword `this`**, **utile per distinguere attributi e variabili locali quando i nomi coincidono e per richiamare costruttori all'interno della stessa classe.**
- **`Static`** e **`final`**, **che definiscono rispettivamente membri condivisi tra tutte le istanze e membri immutabili, con applicazioni pratiche per costanti e metodi utilitari.**
- Inner class, **per raggruppare logicamente classi strettamente correlate e controllare la loro visibilità.**

Questi strumenti ci permettono di modellare gli oggetti in modo **chiaro, modulare e sicuro**, ma da soli non bastano a proteggere lo **stato interno degli oggetti**.
Per garantire che gli attributi siano sempre coerenti e accessibili solo in modi controllati, interviene il concetto di **incapsulamento**, uno dei principi fondamentali della programmazione orientata agli oggetti.

L'**incapsulamento** consiste nel:

- **nascondere i dettagli interni di una classe**, esponendo solo ciò che è necessario tramite **metodi pubblici controllati** (getter e setter).
  In questo modo, la classe diventa più sicura, manutenibile e meno soggetta a errori derivanti da manipolazioni esterne indesiderate.

Incapsulamento e consistenza degli oggetti

Un aspetto fondamentale della programmazione orientata agli oggetti è **mantenere gli oggetti in uno stato consistente**.
La **consistenza** di un oggetto significa che:

- **i suoi attributi contengono valori coerenti e validi nel tempo, secondo le regole della classe.**

In Java, le **operazioni di scrittura sugli attributi** sono particolarmente delicate:

- **modificare direttamente i valori può introdurre incoerenze o violare le regole interne dell'oggetto, compromettendo il funzionamento dell'intero programma**.

Per proteggere lo stato interno degli oggetti interviene l'**incapsulamento**, la tecnica con cui Java consente di **nascondere i dettagli interni** di una classe.
In pratica:

- Gli **attributi vengono resi** `private` → **impedendo l'accesso diretto dall'esterno della classe.**
- **L'accesso e la modifica dei dati avvengono solo tramite metodi pubblici controllati (getter e setter), che possono applicare controlli, trasformazioni o regole aggiuntive.**

Questo meccanismo garantisce che gli oggetti restino **sempre in uno stato valido e coerente**, aumentando la **sicurezza**, la **manutenibilità** e la **robustezza** del codice.

Come realizzare l'incapsulamento

Per proteggere lo **stato interno di un oggetto** e garantire la sua consistenza, in Java si seguono due passaggi principali:

1. **Nascondere gli attributi**
   - **Gli attributi della classe vengono dichiarati** `private`**, impedendo l'accesso diretto dall'esterno.**
   - **In questo modo, nessun codice esterno può modificarli senza passare dai metodi della classe**.
2. **Fornire metodi di accesso controllato**
   - **Getter (metodi di lettura): restituiscono il valore dell'attributo**.

Java

```java
public TipoAttributo getAttributo() {
    return attributo;
}
```

> **NB:**
>
> se l'attributo non è di tipo primitivo, è preferibile **restituire una copia** dell'oggetto e non il riferimento diretto, per evitare modifiche indirette dall'esterno.

- **Setter (metodi di scrittura): permettono di modificare il valore dell'attributo, applicando eventuali controlli**.

Java

```java
public void setAttributo(TipoAttributo valore) {
    this.attributo = valore;
}
```

Questo schema garantisce che **qualsiasi accesso o modifica ai dati sia controllato**, proteggendo l'oggetto da stati inconsistenti e facilitando la manutenzione del codice.

Modificatori di accesso e incapsulamento

Per garantire la **consistenza e la sicurezza degli oggetti**, Java mette a disposizione i **modificatori di accesso**, che stabiliscono **chi può vedere o modificare le classi e i loro membri**.

In pratica, ogni attributo e metodo può avere un livello di visibilità:

- **che determina da dove può essere letto o scritto**.
  Esistono tre modificatori principali e un quarto livello implicito:

1. `public` – **accessibile ovunque**.
2. `protected` – **accessibile alle sottoclassi e alle classi dello stesso package**.
3. `private` – **accessibile solo all'interno della classe che lo dichiara.**
4. ***default* (package-private)** – **accessibile solo alle classi dello stesso package, quando non viene specificato alcun modificatore**.

Nel contesto dell'incapsulamento, i modificatori più rilevanti sono:

- `private` sugli attributi: **impedisce modifiche dirette dall'esterno e forza l'uso di getter e setter.**

- `public` sui metodi di accesso: **consente di leggere o modificare gli attributi in maniera controllata**.

> **In questo modo, i modificatori diventano uno strumento pratico per controllare l'accesso ai dati e proteggere lo stato interno degli oggetti, integrando il concetto di incapsulamento con la gestione della visibilità.**

Visibilità di attributi, metodi e classi

Come già ampiamente detto nelle scorse lezioni, gli attributi e i metodi dichiarati `private` sono:

- **visibili e accessibili solo all'interno della classe in cui sono definiti**. Ciò significa che **non possono essere invocati dall'esterno**, né da altre classi dello stesso package né da sottoclassi:

Java

```java
public class Impiegato {
    private String nome;

    private void stampaNome() {
        System.out.println(nome);
    }
}

public class Main {
    public static void main(String[] args) {
        Impiegato i = new Impiegato();
        // i.nome -> ERRORE: attributo private
        // i.stampaNome() -> ERRORE: metodo private
    }
}
```

> ⚠ **Importante:**
>
> questo comportamento è uno dei pilastri dell'**incapsulamento**, perché protegge lo stato interno dell'oggetto e costringe a utilizzare metodi pubblici controllati (getter e setter) per leggere o modificare gli attributi.

Un'eccezione riguarda le **inner class**:

- **è possibile definire una classe `private` all'interno di un'altra classe.**
- In questo caso, la classe esterna può accedere liberamente ai membri privati della inner class, mentre dall'esterno l' inner class rimane nascosta:

Java

```java
public class Dipartimento {
    private class Stanza {
        private int numero;
    }
}
```

Questa struttura permette di **raggruppare logicamente le classi** e controllarne la visibilità, senza esporre dettagli implementativi all'esterno.

Visibilità predefinita (default o package-private)

Oltre a `private`, `protected` e `public`, Java prevede un quarto livello di visibilità, detto **default**, **package-private** o **impostazione predefinita**.

- Se non specifichi alcun modificatore, **gli attributi e i metodi sono accessibili solo alle classi dello stesso pacchetto che abbiano un riferimento all'oggetto**.
- Per le **classi**, la visibilità default permette che:
  - **siano visibili solo all'interno dello stesso pacchetto, ma non al di fuori.**

Java

```java
class Impiegato { // default (package-private)
    String nome;  // visibile solo nello stesso package
    void stampaNome() { // visibile solo nello stesso package
        System.out.println(nome);
    }
}
```

Nota:

un file Java può contenere più classi con visibilità predefinita, ma al massimo una classe pubblica.

Questo consente di organizzare più classi correlate nello stesso pacchetto senza esporle all'esterno.

In pratica, la visibilità default è utile per

- **raggruppare e proteggere le classi e i membri all'interno di un pacchetto,**
  favorendo l'incapsulamento a livello di pacchetto senza esporre dettagli interni al resto del progetto.

Visibilità `protected`

Il modificatore `protected` estende l'idea di incapsulamento:

- **un membro `protected` è più accessibile di un `private`, ma resta controllato**.
- Gli **attributi e i metodi** `protected` sono visibili a:
  - **Tutte le classi dello stesso pacchetto**
  - **Tutte le sottoclassi, anche se si trovano in pacchetti diversi**

Java

```java
public class Persona {
    protected String nome; // visibile a sottoclassi e classi dello stesso pacchetto
    protected void stampaNome() {
        System.out.println(nome);
    }
}
```

Nota:

non è possibile definire classi `protected` a livello top-level; questa visibilità è consentita solo per le **inner class** all'interno di un'altra classe.

`protected` è particolarmente utile quando progetti una gerarchia di classi:

- **permette alle sottoclassi di accedere ai membri necessari per estendere o modificare il comportamento, senza esporre tutto al mondo esterno.**

Visibilità `public`

Il modificatore `public` rappresenta il **livello massimo di visibilità** in Java.

- **Attributi** `public`
  - **Sono sempre accessibili da qualunque classe, purché si disponga di un riferimento all'oggetto.**
  - **Questo è potenzialmente pericoloso**, perché consente **modifiche dirette allo stato interno dell'oggetto**, con il rischio di **comprometterne la coerenza**.

Java

```java
public class Impiegato {
    public double salario;
}
```

In questo caso, chiunque può scrivere:

Java

```java
impiegato.salario = -1000; // stato incoerente
```

⚠ **Per questo motivo, gli attributi `public` sono fortemente sconsigliati e vanno evitati nella progettazione orientata agli oggetti.**

Metodi `public`

- **Sono sempre visibili da qualsiasi classe che possieda un riferimento all'oggetto**.
- Rappresentano l'**interfaccia pubblica** dell'oggetto, cioè il modo corretto e controllato per interagire con esso.

Java

```java
public void aumentaSalario(double incremento) {
    if (incremento > 0) {
        salario += incremento;
    }
}
```

I metodi `public` sono quindi:

- **lo strumento principale per preservare l'incapsulamento, perché permettono di controllare come lo stato dell'oggetto viene modificato**.

Classi `public`

- Una classe `public` **è visibile da qualsiasi altra classe, sia nello stesso package sia in package diversi**.
- **Un file** `.java` **può contenere al massimo una classe** `public`**, e il nome del file deve coincidere con il nome della classe**.

Java

```java
public class Impiegato {
    // ...
}
```

Collegamento con l'incapsulamento

In sintesi:

- **Attributi** → **dovrebbero essere **`private`**==
- **Metodi** → **generalmente** `public`**, perché definiscono il comportamento dell'oggetto**
- **Classi** → `public` **solo quando devono essere utilizzate dall'esterno del package**

Questo approccio consente di **proteggere lo stato interno degli oggetti** e di garantire che ogni modifica avvenga in modo **controllato e coerente**, che è l'obiettivo principale dell'**incapsulamento in Java**.

Livelli di visibilità in Java

I **modificatori di accesso** definiscono **chi può vedere e utilizzare** classi, attributi e metodi.
In Java esistono **4 livelli di visibilità**, ordinati dal più permissivo al più restrittivo:

- `public` → *massima visibilità*
  - **Il membro è visibile ovunque, da qualsiasi classe e package.**
  - Va usato con attenzione, soprattutto sugli attributi, perché può compromettere l'incapsulamento.

- `protected`
  - Il membro è visibile:
    - **a tutte le classi dello stesso package**
    - **a tutte le sottoclassi, anche se appartenenti a package diversi**
  - È tipico di gerarchie di classi e di progettazione orientata all'ereditarietà.
- **visibilità predefinita (default / package-private)**
  - **Si applica quando non viene specificato alcun modificatore.**
  - **Il membro è visibile solo alle classi dello stesso package.**
  - **Utile per mantenere coesione interna a un package senza esporre dettagli all'esterno.**
- `private` → *minima visibilità*
  - **Il membro è visibile solo all'interno della classe di appartenenza.**
  - **È il modificatore fondamentale per realizzare l'incapsulamento dei dati.**

🔗 Collegamento con l'incapsulamento

Questa gerarchia di visibilità non è casuale:
→ **più un membro è visibile, meno è protetto**.

Per questo, nella progettazione orientata agli oggetti:

- **gli attributi sono quasi sempre** `private`
- **i metodi costituiscono l'accesso controllato allo stato dell'oggetto**
- **la visibilità viene scelta in funzione del ruolo del membro, non per comodità**

In altre parole, i modificatori di accesso sono lo strumento con cui Java permette di **bilanciare accessibilità e sicurezza**, garantendo oggetti **coerenti, robusti e manutenibili nel tempo**.

Buone pratiche di progettazione delle classi

Per rendere le classi **sicure, coerenti e manutenibili**, l'uso corretto dei modificatori deve essere accompagnato da alcune **regole di progettazione fondamentali**.

In primo luogo,

1. gli **attributi dovrebbero essere preferibilmente** `private`.
   - **Questo consente di proteggere lo stato interno dell'oggetto e di impedire modifiche incontrollate dall'esterno, preservandone la consistenza nel tempo.**
2. **È inoltre buona norma inizializzare sempre gli attributi, tipicamente all'interno del costruttore**.
   - **Un oggetto dovrebbe nascere già in uno stato valido e coerente, evitando valori indefiniti o dipendenze implicite**.
3. I **metodi di accesso e modifica (getter e setter)** vanno introdotti **solo quando sono realmente necessari**.
   - **Non tutti gli attributi devono essere esposti:**
     - **rendere disponibile un setter significa permettere una modifica dello stato, e ogni modifica rappresenta un potenziale rischio per la coerenza dell'oggetto.**
4. Un'altra regola importante riguarda l'uso dei **tipi essenziali** (come `int`, `double`, `String`).
   - **Quando un insieme di dati ha un significato concettuale proprio, è spesso preferibile incapsularlo in una nuova classe, invece di spargere tipi primitivi nel codice**.
   - **Questo migliora espressività, controllo e riuso**.
5. **Le classi con troppe responsabilità andrebbero invece suddivise**.
   - **Una classe dovrebbe avere un compito chiaro e ben definito: classi troppo grandi o "onniscienti" sono difficili da comprendere, testare e mantenere**.
6. **Infine, è fondamentale assegnare nomi significativi a classi, attributi e metodi**.
   - **Un buon nome riduce la necessità di commenti e rende il codice auto-esplicativo, facilitando la lettura e l'evoluzione del software**.